The file provider.js exports a function called `importData` to import the example data set from our movies.json file and insert it into our `movies` class, which you can see in the following code snippet:

```
const importData = () => {                          #1
  const data = require('.././data/movies.json');   #1
  data.reduce((a, v) => {   #1
    v.id = a + 1;   #1
    movies.insertMovie(v);   #1
    return a + 1;   #1
  }, 0);   #1
};   #1
```

#1 Load default data into the Movies class

To create the movies API and expose the different endpoints, we've used the Express framework (https://expressjs.com/), a popular framework for creating backend applications in a client-server architecture. Express can be used to create and expose APIs for your clients to use. The provider.js also contains the code to set up the Express framework and the routes needed for the API to respond to the client's request. For example, the following code snippet registers two new `GET` requests to Express:

```
...

const movies = new Movies();

server.get('/movies', (req, res) => {
  res.send(movies.getMovies());
});

server.get('/movie/:id', (req, res) => {
  const movie = movies.getMovieById(req.params.id);
  if (!movie) {
    res.status(404).send('Movie not found');
  } else {
    res.send(movie);
  }
});

...
```

The first route registers that any `GET` requests made to `'/movies'` will respond by sending all the movies. The second route registers `GET` requests to `'/movie:id'`, with the ID as a route parameter. Route parameters capture the values provided at their position in the URL. For example, `'/movie/1'` should return the movie "The Shawshank Redemption" while `'/movie/1000'` should return an error with the message "Movie not found."